

Módulo 9 – Capítulo 01 – Sección 01

La superficie de ataque de los sistemas de IA: qué es nuevo respecto a sistemas tradicionales

En el Módulo 8 aprendimos a desplegar infraestructura de inferencia real: levantamos servidores vLLM y Text Generation Inference (TGI) para servir modelos en producción, configuramos pipelines de MLflow para rastrear artefactos de modelo, indexamos documentos en Pinecone o Weaviate para alimentar sistemas RAG, y conectamos pipelines de fine-tuning al almacenamiento en S3 o GCS. Esa infraestructura que construimos con tanto cuidado —el servidor vLLM escuchando en el puerto 8000, el índice vectorial actualizado periódicamente, el repositorio de modelos en MLflow— es exactamente la superficie de ataque que este módulo analiza. Pasar del Módulo 8 al Módulo 9 no es un salto temático: es girar la perspectiva ciento ochenta grados, del diseñador al adversario.

Los sistemas de IA introducen vectores de ataque que no existen en el software tradicional, y la diferencia es estructural, no incidental. En una API REST convencional, el servidor ejecuta lógica determinista: la misma entrada produce siempre la misma salida, y las vulnerabilidades son bugs en el código que pueden ser encontrados y corregidos con análisis estático. Un LLM como GPT-4o o Claude 3.5 Sonnet, en cambio, genera outputs probabilísticos basados en instrucciones en lenguaje natural. Este diseño hace que el modelo en sí mismo sea una superficie de ataque activa: puede ser inducido a cambiar su comportamiento mediante texto cuidadosamente construido, sin necesidad de explotar ninguna vulnerabilidad en el código. Un firewall de red no puede bloquear un prompt injection porque el payload malicioso llega en el mismo canal que las instrucciones legítimas; un WAF tradicional no entiende la semántica de un jailbreak porque el ataque no tiene firma a nivel de protocolo HTTP.

A esta superficie de ataque del modelo se suman las superficies de los componentes que lo rodean. El embedding space de los modelos de representación no tiene equivalente en sistemas CRUD: los vectores almacenados en Pinecone, Weaviate, Chroma o pgvector no son solo datos inertes sino representaciones semánticas que determinan qué información recibe el modelo como contexto. Un documento malicioso en el índice vectorial no rompe el sistema en el sentido técnico clásico, pero puede modificar el comportamiento del modelo de maneras

que una query de SQL injection nunca podría. De manera similar, el pipeline de fine-tuning — que en el Módulo 8 configuramos con tanta atención a la calidad de los datos— es una superficie de ataque si los datos de entrenamiento pueden ser contaminados por un adversario.

Comprender esta expansión de la superficie de ataque es el prerequisite para cualquier decisión de seguridad que se tome en los capítulos siguientes. No se trata de aplicar las mismas soluciones de seguridad web a un contexto diferente; se trata de reconocer que los sistemas de IA requieren un modelo de amenazas propio.

Componentes de la nueva superficie de ataque

- **Model input surface:** el prompt —en texto, imágenes, audio o documentos— es un vector de entrada que no puede sanitizarse mediante reglas tradicionales de WAF porque la "malicia" está en el significado semántico del texto, no en su estructura sintáctica o protocolar.
- **Model weights y fine-tuning pipeline:** los pesos del modelo pueden ser adulterados durante el fine-tuning si el dataset de entrenamiento está contaminado con ejemplos maliciosos, modificando el comportamiento del modelo en producción de maneras latentes y difíciles de detectar.
- **Retrieval layer (RAG):** los índices vectoriales y los documentos recuperados son superficies de inyección de instrucciones maliciosas que el modelo ejecutará como parte de su contexto, especialmente cuando el corpus incluye documentos de fuentes externas no verificadas.
- **Tool-calling interface:** en sistemas agénticos contruidos con LangChain, LangGraph o CrewAI, las herramientas expuestas al modelo —ejecutar código, llamar APIs, leer archivos, enviar emails— amplifican exponencialmente el impacto de cualquier ataque exitoso.
- **Output processing pipeline:** las respuestas del modelo pueden contener código malicioso, instrucciones de rendering o rutas de exfiltración de datos si se renderizan en el frontend sin sanitización apropiada.
- **Serving layer:** el servidor vLLM, TGI, o Triton Inference Server que desplegamos en el Módulo 8 tiene sus propias vulnerabilidades de infraestructura —escapes de container, exploits de la API de administración, escalada de privilegios— que deben gestionarse con los mismos controles que cualquier servidor de aplicaciones crítico.

Nota del Arquitecto: *En producción, las vulnerabilidades más frecuentes no están en el modelo sino en la capa de aplicación que lo rodea: concatenar el input del usuario directamente en el prompt sin separación estructural, almacenar API keys en el system prompt, o indexar en el vectorstore documentos de usuarios sin validación de contenido. Estos errores de diseño ocurren porque los equipos tratan al LLM como una API más, sin reconocer que su input es también su "código ejecutable".*

La superficie de ataque de un sistema de IA incluye cada capa del pipeline —desde el input del usuario hasta los pesos del modelo, pasando por el serving layer, el vectorstore y las herramientas del agente— y cada capa debe tener controles de seguridad independientes porque ninguna capa puede ser considerada confiable por sí sola. Esta es la premisa sobre la que se construye todo el módulo.

Módulo 9 – Capítulo 01 – Sección 02

Taxonomía de amenazas: input attacks, model attacks, supply chain attacks y data attacks

Sin un vocabulario compartido para categorizar las amenazas, los controles de seguridad se aplican de forma reactiva: se parchea la última vulnerabilidad conocida sin entender si el control corresponde a la capa donde el ataque opera. La taxonomía de amenazas en sistemas de IA es el mapa que permite razonar sistemáticamente sobre qué puede salir mal, en qué capa y con qué controles preventivos. OWASP LLM Top 10 (versión 2023-2024) y el marco MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) proporcionan las dos taxonomías complementarias más consolidadas: OWASP LLM Top 10 enumera las vulnerabilidades de mayor prevalencia en aplicaciones basadas en LLMs; MITRE ATLAS describe las tácticas, técnicas y procedimientos (TTPs) de los atacantes organizados en una matriz análoga a MITRE ATT&CK pero específica para sistemas de machine learning.

La taxonomía que utilizaremos a lo largo de este módulo organiza las amenazas en cinco categorías según la capa del sistema donde el ataque opera, porque cada categoría requiere controles en una capa diferente. Los input attacks operan en tiempo de inferencia y se mitigan en el gateway de la aplicación y en el diseño del prompt. Los model attacks comprometen los pesos o el proceso de entrenamiento y se mitigan en el pipeline de ML. Los supply chain attacks afectan las dependencias del sistema antes de que lleguen al entorno de producción y se mitigan en los procesos de adquisición y verificación de componentes. Los data attacks apuntan a los datos que el modelo consume como contexto y se mitigan en la pipeline de ingestión. Los infrastructure attacks explotan el serving layer y la plataforma de despliegue y se mitigan con hardening de infraestructura convencional.

La categoría de supply chain attacks merece atención especial porque el Módulo 8 enseñó exactamente las operaciones que estos ataques comprometen: descargar modelos de Hugging Face, instalar paquetes de ML desde PyPI, y cargar datasets desde repositorios públicos. Cuando un modelo en Hugging Face tiene sus pesos modificados mediante pickle exploitation, o cuando una dependencia maliciosa en `torch` o `transformers` se instala silenciosamente en el entorno de fine-tuning, el ataque ocurre antes de que el sistema de IA

haya procesado un solo request. Esta superficie de ataque es especialmente insidiosa porque los equipos que siguen buenos estándares de seguridad en la capa de aplicación pueden ser completamente vulnerables a compromisos en la capa de adquisición de modelos. En el Capítulo 03 Sección 05 de este módulo profundizaremos en las técnicas específicas de supply chain security para ML.

Categorías de amenazas en sistemas de IA

- **Input attacks:** prompt injection directa e indirecta, jailbreaking, prompt leaking y ataques de denegación de servicio mediante prompts de alta complejidad computacional (context-flooding con ventanas de 128k tokens que generan costos de procesamiento desproporcionados). Son las amenazas de mayor prevalencia en producción y el foco del Capítulo 02.
- **Model attacks:** model extraction via consultas sistemáticas a la API para reconstruir el comportamiento del modelo propietario, membership inference para detectar si un dato específico fue parte del entrenamiento, y fine-tuning adversarial para insertar backdoors activados por tokens trigger. Tienen menor prevalencia que los input attacks pero mayor impacto potencial sobre la propiedad intelectual y la privacidad. Capítulo 03.
- **Supply chain attacks:** descarga de modelos con pesos modificados desde Hugging Face mediante pickle exploitation, dependencias maliciosas en packages de ML (torch, transformers, accelerate, peft), datasets adulterados en pipelines de data ingestion, y uso de model cards fraudulentas que describen incorrectamente el comportamiento del modelo. El atacante compromete el sistema antes de que entre en producción.
- **Data attacks:** RAG poisoning mediante inyección de documentos maliciosos en Pinecone, Weaviate o Chroma que el modelo recuperará como contexto confiable, y training data poisoning para degradar la accuracy del modelo o insertar comportamientos backdoor específicos activados por triggers. Capítulo 05.
- **Infrastructure attacks:** explotar vulnerabilidades en el serving layer (Triton Inference Server, vLLM, TGI), escalar privilegios desde el contexto del modelo hacia el host del container, o comprometer el pipeline de MLflow para sustituir modelos legítimos por modelos maliciosos sin que el equipo lo detecte.

Nota del Arquitecto: El error más frecuente en threat modeling de sistemas de IA es enfocarse exclusivamente en los input attacks (prompt injection, jailbreaking) porque son los más visibles, e ignorar los supply chain attacks porque son los más

invisibles. En producción, un equipo que tiene excelentes guardrails contra prompt injection pero descarga modelos de Hugging Face sin verificar checksums o sin escanear con herramientas como `modelscan` tiene un perfil de seguridad inconsistente que los adversarios sofisticados explotan precisamente porque nadie lo mira.

Mapear cada amenaza a esta taxonomía antes de diseñar controles es el prerrequisito de un programa de seguridad coherente. Cada categoría de amenaza requiere controles en una capa diferente del sistema: los input attacks se mitigan en el gateway, los model attacks en el pipeline de ML, los supply chain attacks en los procesos de adquisición de dependencias y modelos, los data attacks en la pipeline de ingestión de datos, y los infrastructure attacks en el hardening del serving layer y la plataforma de despliegue. Con este mapa claro, la siguiente sección introduce el marco conceptual de la CIA Triad reinterpretado para sistemas de IA.

Módulo 9 – Capítulo 01 – Sección 03

CIA Triad aplicada a IA: confidencialidad, integridad y disponibilidad en sistemas LLM

La CIA Triad —Confidentiality, Integrity, Availability— es el triángulo fundacional sobre el que se construye cualquier análisis de seguridad de la información. Sin embargo, su aplicación a sistemas de LLM exige reinterpretar cada vértice del triángulo de maneras que van más allá de las definiciones que aprendimos en el contexto de bases de datos relacionales o aplicaciones web. El modelo de lenguaje no es solo un sistema que procesa datos: es en sí mismo un repositorio de información derivada de su corpus de entrenamiento, y esa doble naturaleza —procesador y almacén implícito de datos— crea dimensiones de seguridad que las definiciones clásicas de CIA no anticipan.

La **confidencialidad** en sistemas clásicos refiere principalmente a los datos en tránsito (protegidos con TLS) y en reposo (protegidos con cifrado AES-256). En un sistema LLM, la confidencialidad incluye estas dimensiones convencionales pero añade una tercera que no tiene equivalente en sistemas CRUD: la información que el modelo puede haber memorizado de su corpus de entrenamiento. GPT-2 y GPT-3.5 fueron demostrados por Carlini et al. reproduciendo verbatim secuencias del corpus de pretraining, incluyendo nombres, direcciones de correo electrónico y fragmentos de código propietario. Este riesgo es especialmente relevante cuando una organización hace fine-tuning de un modelo base con datos internos: el modelo resultante puede revelar esos datos internos a usuarios que no deberían tener acceso a ellos, a través de preguntas diseñadas para provocar la memorización. La confidencialidad en IA cubre no solo los datos almacenados sino la memoria implícita del modelo.

La **integridad** en sistemas clásicos significa que los datos no han sido modificados de forma no autorizada. En sistemas LLM, la integridad tiene una dimensión adicional: la consistencia del comportamiento del modelo con sus especificaciones de diseño. Un atacante que logra insertar un backdoor durante el fine-tuning no modifica los pesos del modelo de manera que sea detectável como "corrupción de datos" en el sentido clásico; el modelo resultante pasará la mayoría de los tests de calidad y se comportará normalmente el 99.9% del tiempo, pero

producirá outputs específicos y controlados por el atacante cuando el input contiene el token trigger predefinido. Esta violación de la integridad del comportamiento es tan seria como una corrupción de datos clásica, pero requiere técnicas de detección completamente diferentes: no checksums de archivos sino auditorías de comportamiento del modelo.

La **disponibilidad** en sistemas LLM incluye la resistencia a ataques específicos del modelo: prompts diseñados para generar respuestas de máxima longitud, activar bucles de razonamiento en chain-of-thought, o explotar la ventana de contexto completa (128k tokens en modelos modernos) con el objetivo de agotar la capacidad de cómputo del sistema y denegar el servicio a usuarios legítimos. Un prompt de 100k tokens bien construido puede generar latencias de decenas de segundos y costos computacionales desproporcionados en un sistema sin rate limiting por tokens. Estos ataques de prompt-DoS no tienen equivalente en APIs REST convencionales donde el costo de procesar una request es relativamente uniforme.

Aspectos técnicos de la CIA en sistemas LLM

- **Confidencialidad del model behavior:** los pesos del modelo pueden revelar datos de entrenamiento mediante ataques de extracción; el system prompt puede ser filtrado mediante prompt leaking; las respuestas pueden contener PII memorizada del corpus de pretraining, incluso en modelos desplegados para tareas completamente diferentes.
- **Confidencialidad del pipeline de MLOps:** los artefactos de modelo en MLflow, los datasets en S3/GCS y los logs de entrenamiento contienen información tan sensible como los datos que procesaron, y deben protegerse con controles de acceso equivalentes a los de una base de datos de producción con datos personales.
- **Integridad del modelo:** el comportamiento del modelo debe ser predecible y consistente con sus especificaciones; un backdoor insertado durante fine-tuning compromete la integridad funcional aunque los pesos no estén corruptos en el sentido binario clásico.
- **Integridad de los datos de inferencia:** los documentos en el vectorstore (Pinecone, pgvector, Weaviate) deben provenir de fuentes validadas; un documento malicioso en el índice compromete la integridad del contexto que el modelo usa para generar respuestas.
- **Disponibilidad frente a prompt attacks:** prompts diseñados para maximizar el uso de la ventana de contexto o generar respuestas de longitud extrema son ataques de DoS específicos de LLMs que requieren controles de rate limiting por tokens, no solo por requests.

Nota del Arquitecto: *El error conceptual más común al aplicar CIA a sistemas de IA es subestimar la confidencialidad del modelo mismo como activo de información. Las organizaciones cifran meticulosamente sus bases de datos de clientes y luego hacen fine-tuning de modelos con esos mismos datos sin differential privacy, creando un nuevo activo —el modelo fine-tuneado— que puede revelar los datos originales. El modelo es una base de datos implícita y debe protegerse como tal.*

La CIA Triad aplicada a sistemas LLM exige extender las definiciones tradicionales para cubrir el modelo como un activo de información en sí mismo, cuya confidencialidad (memorización), integridad (comportamiento consistente) y disponibilidad (resistencia a prompt-DoS) deben protegerse con los mismos controles rigurosos que cualquier base de datos crítica. Esta extensión conceptual es el fundamento sobre el que se construye el modelado de amenazas de la siguiente sección.

Módulo 9 – Capítulo 01 – Sección 04

Modelado de amenazas para aplicaciones de IA: STRIDE y AI-specific frameworks

El modelado de amenazas es la actividad que convierte la intuición sobre riesgos en un inventario estructurado y priorizado de amenazas concretas. Sin este proceso, un equipo de AI Engineering aplica controles de seguridad en los lugares donde más los teme — frecuentemente los más visibles, como el endpoint de inferencia— mientras deja sin cubrir superficies de ataque igualmente críticas como el pipeline de MLflow o los permisos de escritura en el índice vectorial. El threat modeling transforma la seguridad de una colección de decisiones ad hoc en un programa sistemático donde cada control puede trazarse hasta una amenaza específica en el modelo, y cada brecha de control puede identificarse antes de que un adversario la explote.

STRIDE —Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege— es el framework desarrollado por Microsoft que organiza el espacio de amenazas en seis categorías cubren la mayoría de las situaciones que un equipo de ingeniería enfrenta en producción. Aplicado a sistemas de IA, cada categoría adquiere manifestaciones específicas que no existían en las versiones originales del framework: el Spoofing en un LLM puede ser un atacante que hace pasar instrucciones maliciosas como si vinieran del operador del sistema mediante prompt injection; el Tampering puede ser la modificación del comportamiento del modelo a través de data poisoning durante el fine-tuning; la Repudiation se manifiesta en la falta de logs inmutables que impiden atribuir con certeza qué output produjo el modelo ante qué input; la Information Disclosure cubre tanto el prompt leaking clásico como la memorización de datos sensibles del corpus de entrenamiento; el DoS incluye context-flooding con prompts de 128k tokens; y la Elevation of Privilege ocurre cuando un jailbreak exitoso permite al usuario hacer que el agente ejecute acciones que sus privilegios no deberían permitir.

MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) complementa STRIDE con una perspectiva orientada al adversario: en lugar de categorizar amenazas por tipo, organiza los ataques por las tácticas que los atacantes usan —ML Attack

Staging, Model Evasion, Model Theft, Data Poisoning, entre otras— con técnicas documentadas y ejemplos reales de ataques observados en producción. ATLAS es análogo a MITRE ATT&CK para sistemas de IA: proporciona el vocabulario que los equipos de seguridad usan para comunicar amenazas de forma inequívoca y para mapear observaciones de producción a técnicas de ataque conocidas. La combinación de STRIDE (para el razonamiento sistemático por categoría de amenaza) y ATLAS (para el razonamiento orientado al adversario) es más completa que cualquiera de los dos frameworks por separado.

El elemento técnico más importante del proceso de threat modeling es el Data Flow Diagram (DFD) con trust boundaries explícitas. Un DFD de sistema RAG típico muestra cinco componentes: el usuario (fuera de confianza), el gateway de la aplicación, el LLM, el vectorstore, y las herramientas externas. Cada flecha entre componentes que cruza una trust boundary es un punto donde STRIDE debe aplicarse sistemáticamente: la flecha del usuario al gateway cruza la boundary "público → sistema confiado" y es vulnerable a Spoofing e Information Disclosure; la flecha del vectorstore al LLM cruza la boundary "datos externos → modelo" y es vulnerable a Tampering (documentos maliciosos) e Information Disclosure (datos de otros tenants). Aplicar STRIDE en cada cruce de boundary genera una lista completa de amenazas que puede priorizarse con frameworks como DREAD (Damage, Reproducibility, Exploitability, Affected users, Discoverability).

Componentes del modelado de amenazas para IA

- **STRIDE aplicado a LLMs:** Spoofing del identity del sistema via prompt injection que hace pasar instrucciones del atacante como instrucciones del operador; Tampering de outputs via manipulación del contexto RAG; Repudiation por ausencia de logs inmutables de inferencia; Information Disclosure via prompt leaking o memorización; DoS via context-flooding; Elevation of Privilege via jailbreak que desbloquea capacidades restringidas.
- **MITRE ATLAS TTPs:** las 14 tácticas incluyen ML Attack Staging (preparar el ataque mediante reconnaissance del sistema), Model Evasion (eludir clasificadores de contenido), Model Theft (extraer el modelo propietario via queries sistemáticas), y Data Poisoning (contaminar el corpus de entrenamiento); cada táctica tiene técnicas documentadas con casos reales.
- **Data Flow Diagrams para IA:** el DFD debe incluir el modelo como componente con trust boundaries explícitas entre usuario, gateway, LLM, vectorstore y herramientas externas; los documentos del corpus RAG y los resultados de herramientas son entidades de datos en el DFD con su propio nivel de confianza.

- **AI-specific trust boundaries:** el prompt del usuario (untrusted), los documentos recuperados por RAG (semi-trusted según el origen), los resultados de herramientas (trusted solo si el tool está bajo control del sistema), y el historial de conversación (mixed-trust porque incluye tanto mensajes del sistema como del usuario).
- **Threat enumeration y priorización:** aplicar STRIDE a cada cruce de trust boundary genera la lista completa de amenazas; priorizarlas con DREAD o CVSS adaptado permite enfocar los controles en las amenazas de mayor impacto y probabilidad.

Nota del Arquitecto: El DFD más incompleto que veo en revisiones de arquitectura es el que no incluye el pipeline de fine-tuning como superficie de ataque. Los equipos dibujan el sistema de inferencia con detalle, pero el proceso de entrenamiento —el S3 bucket con los datos de entrenamiento, el script de fine-tuning, el endpoint de MLflow donde se registra el modelo— queda fuera del diagrama y por tanto fuera del threat model. Un atacante con acceso de escritura al bucket de training data puede comprometer el modelo resultante sin tocar nunca el sistema de inferencia.

El threat model de un sistema de IA debe actualizarse cada vez que se agrega una nueva herramienta al agente, se cambia el modelo base, o se expande el corpus del vectorstore. Cada cambio de capacidad es un cambio en la superficie de ataque que invalida parcialmente el modelo de amenazas anterior. Esta necesidad de actualización continua es el puente hacia la siguiente sección, que trata la responsabilidad compartida: quién debe actualizar qué parte del threat model cuando el proveedor del modelo base lanza una nueva versión.

Módulo 9 – Capítulo 01 – Sección 05

Responsabilidad compartida: proveedor de modelo, plataforma y aplicación

El modelo de responsabilidad compartida es uno de los conceptos más clarificadores de la arquitectura de seguridad cloud: establecer explícitamente quién es responsable de qué evita tanto la duplicación de controles como, más peligrosamente, la suposición de que alguien más está protegiendo una superficie que en realidad nadie protege. En la nube convencional (AWS, Azure, GCP), el modelo es claro: el proveedor protege la infraestructura física, el hypervisor y los servicios gestionados; el cliente protege los sistemas operativos, las aplicaciones y los datos. En sistemas de IA, el modelo de responsabilidad tiene una capa adicional —el proveedor del modelo base— y los límites entre capas son más difusos, lo que genera confusión sistemática sobre qué controles son provistos y cuáles deben ser implementados por el equipo de desarrollo.

El **proveedor del modelo base** (OpenAI, Anthropic, Google, Meta) es responsable de la seguridad del proceso de pretraining y RLHF, de los controles de seguridad embebidos en el modelo (Constitutional AI en Anthropic, los sistemas de seguridad de content policy en OpenAI), de la protección de la infraestructura de entrenamiento, y de la divulgación responsable de vulnerabilidades conocidas del modelo. Esta responsabilidad no es auditable ni modificable por el desarrollador: cuando usamos GPT-4o o Claude 3.5 Sonnet, estamos confiando en los controles de seguridad que el proveedor declara en sus model cards y políticas de uso, sin capacidad de verificarlos directamente. Los controles del proveedor son el piso de seguridad del modelo base, y el piso puede tener grietas que el proveedor aún no ha descubierto o parchado.

La **plataforma de serving** (Azure OpenAI, AWS Bedrock, Vertex AI) gestiona la infraestructura de inferencia, el aislamiento de tenants en entornos multi-tenant, los controles de red, el cifrado en tránsito, y los SLAs de disponibilidad. El desarrollador configura estos controles (parámetros de rate limiting, regiones de datos, modelos disponibles) pero no los implementa desde cero. Las plataformas de serving también tienen sus propias vulnerabilidades históricas: en 2023, se detectaron vulnerabilidades en la API de

Azure OpenAI que permitían cierta visibilidad cruzada entre tenants, un recordatorio de que incluso la infraestructura gestionada no es perfecta.

El **desarrollador de la aplicación** es responsable de todo lo que ocurre en la capa de aplicación, y en la práctica esta capa contiene la gran mayoría de las vulnerabilidades en producción: prompt design inseguro que no previene prompt injection, ausencia de guardrails adicionales sobre los del proveedor, control de acceso insuficiente a endpoints del sistema, logging inadecuado, y manejo inseguro de PII en el contexto de la conversación. La razón por la que la capa de aplicación concentra la mayoría de los incidentes no es que los proveedores sean más competentes, sino que la capa de aplicación es la única capa que el equipo de desarrollo controla completamente y donde las decisiones de seguridad se toman (o evitan) deliberadamente.

Puntos críticos del modelo de responsabilidad

- **Proveedor de modelo base:** seguridad de los pesos durante el entrenamiento, controles de alignment (RLHF, Constitutional AI), filtros de contenido embebidos en el modelo, y protección de la infraestructura de entrenamiento. El desarrollador no puede auditar ni modificar estos controles; debe evaluarlos a través de la documentación pública y model cards del proveedor.
- **Proveedor de plataforma (API):** rate limiting a nivel de plataforma, autenticación de API keys, TLS en tránsito, aislamiento de requests entre clientes, y SLA de disponibilidad. El desarrollador configura estos controles dentro de los parámetros que la plataforma expone, pero no implementa la infraestructura subyacente.
- **Desarrollador de la aplicación:** diseño seguro del system prompt, validación y sanitización de user inputs, implementación de guardrails adicionales (LlamaGuard, Azure Content Safety) por encima de los del proveedor, control de acceso a endpoints, logging de seguridad, y manejo seguro de PII en el contexto de conversación y en los datos de fine-tuning.
- **Zonas grises críticas:** la prompt injection indirecta via documentos recuperados por RAG es responsabilidad del desarrollador aunque el modelo sea el ejecutor; las respuestas inapropiadas en contextos específicos del dominio (por ejemplo, un asistente médico que da consejo clínico incorrecto) requieren guardrails adicionales a nivel de aplicación aunque el modelo base no las genere en contextos generales.
- **Fine-tuning y modelos propios:** cuando el equipo realiza fine-tuning, la responsabilidad de la seguridad del modelo derivado recae completamente en el equipo, incluyendo la protección del pipeline de entrenamiento contra data poisoning, la

implementación de differential privacy si los datos de entrenamiento son sensibles, y la auditoría del comportamiento del modelo resultante antes del despliegue.

Nota del Arquitecto: *El error de seguridad más costoso que veo en auditorías de sistemas de IA es el que llamo "responsabilidad implícita": el equipo asume que el proveedor del modelo protege contra jailbreaks, o que la plataforma de serving gestiona la autenticación a nivel de usuario final, o que los guardrails del proveedor son suficientes para el dominio específico de la aplicación. En sistemas de atención médica, jurídica o financiera, los guardrails genéricos del proveedor no son suficientes: la aplicación debe implementar guardrails específicos del dominio porque el proveedor no puede anticipar todos los contextos en los que su modelo será desplegado.*

En un sistema de IA en producción, la seguridad que el proveedor del modelo garantiza es el piso mínimo, no la protección completa. El Capítulo 02 comienza precisamente donde termina la responsabilidad del proveedor: las vulnerabilidades de prompt injection y jailbreaking que los controles del modelo no pueden prevenir por sí solos, y que el desarrollador debe abordar con controles propios en la capa de aplicación.

Módulo 9 – Capítulo 01 – Sección 06

Cierre: la seguridad en IA comienza por entender que el modelo es también una superficie de ataque

La seguridad en sistemas de IA no es una extensión de la seguridad de aplicaciones web: es una disciplina que requiere reconceptualizar qué significa "atacar" un sistema cuando el componente central no ejecuta código determinista sino que genera outputs probabilísticos basados en instrucciones en lenguaje natural. Un firewall de red no puede bloquear un prompt injection porque el payload malicioso llega dentro de texto semánticamente válido en el protocolo HTTP. Un WAF tradicional no detecta un jailbreak porque la "firma" del ataque no está en los bytes de la request sino en la intención comunicada en el lenguaje natural. Un IDS convencional no identifica model extraction porque los patrones de queries que recopilan el comportamiento del modelo son indistinguibles, en la capa de red, de un usuario que simplemente hace muchas preguntas.

Esta asimetría entre las herramientas de seguridad disponibles y la naturaleza de las amenazas específicas de IA es la razón por la que el módulo comienza con un capítulo conceptual antes de entrar en controles específicos. Los fundamentos establecidos aquí — superficie de ataque expandida que incluye el modelo, el vectorstore, el pipeline de fine-tuning y el serving layer; taxonomía de amenazas en cinco categorías (input, model, supply chain, data, infrastructure); CIA Triad reinterpretada para cubrir la memorización del modelo, la integridad del comportamiento y el prompt-DoS; threat modeling con STRIDE y ATLAS sobre DFDs con trust boundaries explícitas; y modelo de responsabilidad compartida con tres capas— son el marco conceptual sobre el que se construyen todos los controles técnicos de los capítulos siguientes.

Sin este marco, los controles se aplican de forma ad hoc: se añade un guardrail aquí porque hubo un incidente, se endurece el rate limiting allá porque un cliente se quejó, se añade logging cuando un regulador lo exige. El resultado es un sistema con controles desconectados que dejan brechas sistemáticas que los adversarios explotan de manera predecible, precisamente porque los equipos sin marco conceptual tienden a proteger las mismas superficies visibles mientras ignoran las mismas superficies invisibles. El Capítulo 02 entra de

inmediato en la amenaza de mayor prevalencia en producción —prompt injection directa e indirecta— armados con el mapa conceptual que este capítulo ha establecido.

Conceptos clave del capítulo

- **Superficie de ataque de IA:** incluye pesos del modelo, prompts, vectorstores, tool interfaces, pipelines de entrenamiento y serving layer (vLLM, TGI, Triton), todas superficies sin equivalente directo en sistemas de software tradicional.
- **Taxonomía operativa de amenazas:** cinco categorías (input, model, supply chain, data, infrastructure) que mapean a capas específicas del sistema y a controles en esas capas; OWASP LLM Top 10 y MITRE ATLAS como vocabulario común.
- **CIA Triad extendida:** confidencialidad del modelo (memorización de PII del corpus de pretraining), integridad del comportamiento (backdoors que no corrompen bytes pero sí el comportamiento), y disponibilidad ante prompt-DoS (context-flooding con ventanas de 128k tokens).
- **Threat modeling iterativo:** STRIDE aplicado a DFDs con trust boundaries explícitas entre las capas del sistema de IA, actualizado obligatoriamente ante cada cambio de capacidad, mantenido como código versionado en el repositorio del proyecto.
- **Responsabilidad compartida clara:** el proveedor del modelo protege los pesos y el alignment; la plataforma de serving protege la infraestructura; el desarrollador protege todo lo demás —el system prompt, los guardrails adicionales, la validación de inputs y outputs, el logging, el manejo de PII.

Idea central

Entender la seguridad en IA como una disciplina específica —no como una extensión de la seguridad web— es el prerequisite para construir sistemas que resistan ataques reales en producción. Los controles que funcionan para APIs REST convencionales son necesarios pero no suficientes: los sistemas de IA requieren controles adicionales específicos para las amenazas que emergen del comportamiento probabilístico del modelo, de la semántica del lenguaje natural como vector de ataque, y de la naturaleza de los datos que fluyen por el pipeline.

"Security is a process, not a product." — Bruce Schneier, criptógrafo y autor de "Secrets and Lies", articulando el principio que unifica todos los controles de este módulo: la seguridad no

es un estado que se alcanza con la instalación del producto correcto, sino un proceso continuo de evaluación, mejora y adaptación ante adversarios que también evolucionan.